



Lecture[07]

LIST



Ex

- Address book
 - Design an address book to manage personal contact information. A piece of personal contact information includes: name, age, institute, cell-phone number, e-mail, address.
 - The following basic operations should be implemented:
 - (1) Add a new piece of personal contact information.
 - (2) Delete an existing piece of personal contact information.
 - (3) Search a specified piece of personal contact information.
 - (4) Modify an existing piece of personal contact information.
 - (5) Print out all the personal contact information.



Agenda

- What is a List (列表)
 - Conception
 - Implementation (as data structure)
- Array-Based List
- Linked List
 - Structure node
 - Structure LinkList
 - Pointer LINKLIST
 - Operation implementation



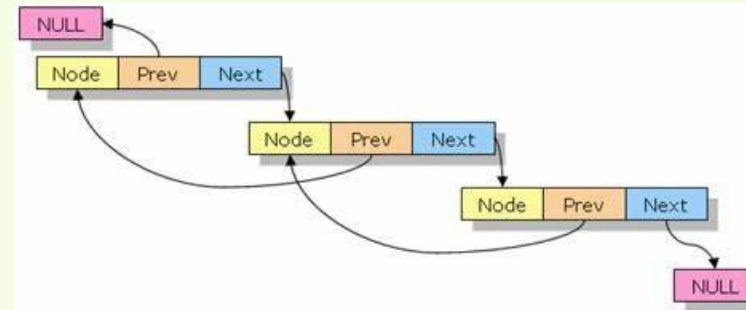
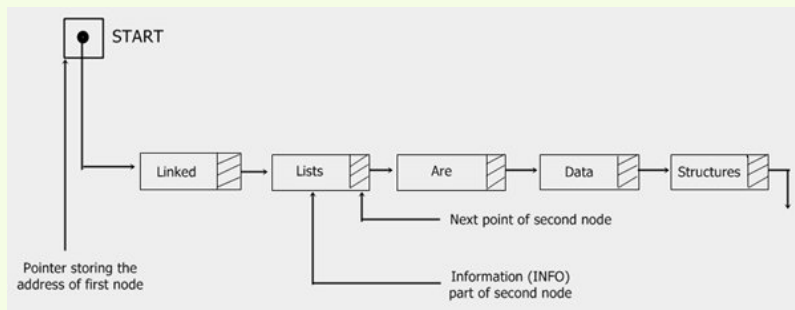
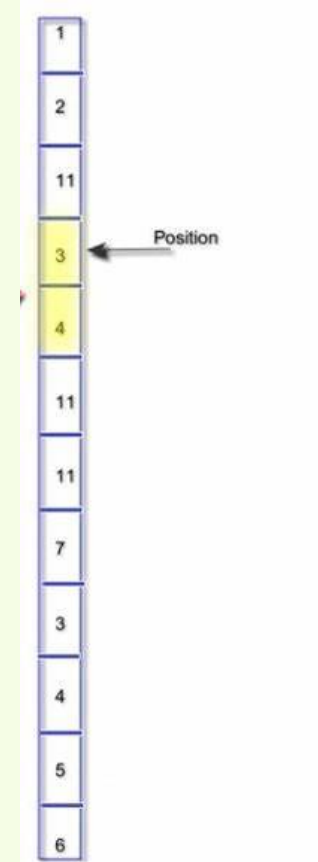
What is a list?

- A number of connected items or names written or printed consecutively, typically one below the other.
- A list is a finite, ordered sequence of data items.
- Important concept: List elements have a position.
- Notation: $\langle a_0, a_1, \dots, a_{n-1} \rangle$
- Size of a list:
 - n
 - Null list: size 0



Array vs List

- Array is a collection of data (same data type) - storage
- List is an abstract data type – **storage + operation**
- We can use an array to implement a list data structure: Array-based list
- But we can have other way to implement a list data structure: linked list (singly linked list, doubly linked list, circular linked list) ...
- A data structure with **one dimension!**





What operations should we implement with a List?

- insert
- getValue
- delete
- find
- clear
- print



List Concepts

Our list implementation will support the concept of a current position.

We will do this by defining the list in terms of left and right partitions.

- Either or both partitions may be empty.

Partitions are separated by the fence (栅栏) .

<20, 23 | 12, 15>



List ADT

```
void clear(List);  
int insert(List, Elem);  
int append(List, Elem);  
int remove(List, Elem*);  
void setStart(List);  
void setEnd(List);  
void prev(List); //把当前位置往前挪一位  
void next(List); //把当前位置往后挪一位  
int setPos(List, int); //设置第i个位置为当前位置  
int getValue(List, Elem*); //取某一项的值  
int leftLength(List);  
int rightLength(List);  
void print(List);
```




List Find Function

Pseudocode

```
// Return true iff K is in list
int find(List L, int K) {
    int it;
    for (setStart(L); getValue(L, &it); next(L))
        if (K == it) return 1;
    return 0;
}
```



List ADT Examples

List: <12 | 32, 15>

```
insert(myList, 99);
```

Result: <12 | 99, 32, 15>

Iterate through the whole list:

```
for (setStart(myList); getValue(myList, &it); next(myList))  
    DoSomething(it);
```

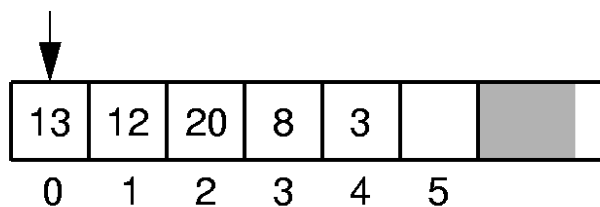


List data structure

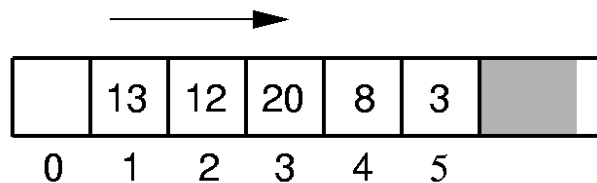
- Array-based list

- Use an array with maxSize to store elements
- Use listSize to specify the length of the list
- Use fence to control the current position
- listArray is the name of the array (pointer)
- AList is the pointer type to the data structure

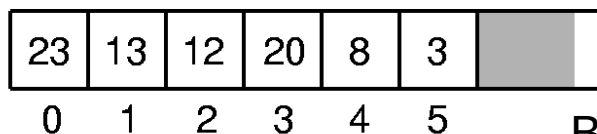
Insert 23:



(a)



(b)



(c)

Best case: 0

Worst case: $O(n)$

```
#define MAX_SIZE 100
```

```
struct alist {
```

```
    int maxSize;
```

```
    int listSize;
```

```
    int fence;
```

```
    int* listArray;
```

```
};
```

```
typedef struct alist* AList
```

```
AList aL=(AList)malloc(sizeof(struct alist));
```

```
aL->maxSize=MAX_SIZE;
```

```
aL->listSize=0;
```

```
aL->fence=0;
```

```
aL->listArray = (int*)malloc(MAX_SIZE*sizeof(int));
```



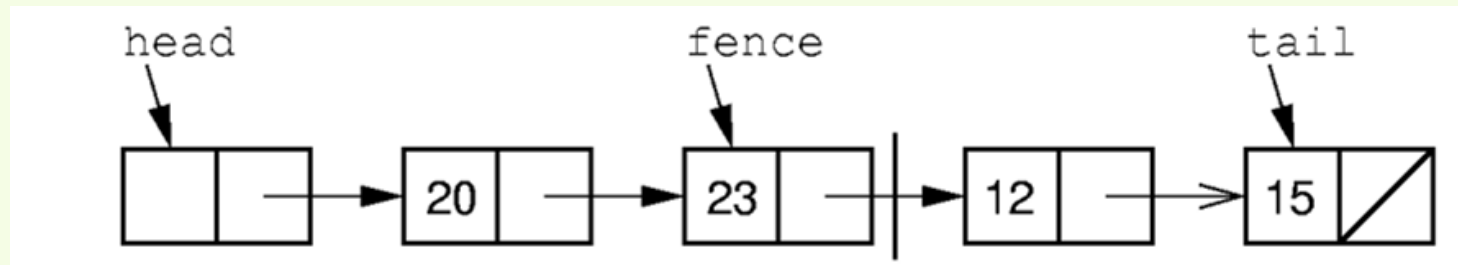
Array-Based List

- All operation could be realized using array
- Need an estimation of maximum size
 - Limitation for unknown size list
 - Space consuming
- `getValue` in $O(1)$
- `find` in $O(n)$
- Insertion and deletion are expensive
 - Move all the elements on the “right”
 - Worst case $O(n)$



List data structure

- Linked list
 - Each node is a structure with self-reference
 - MaxSize and listSize is not necessary
 - head, tail, fence are used as node pointer
 - Use a pointer to the structure





Linked List (1)

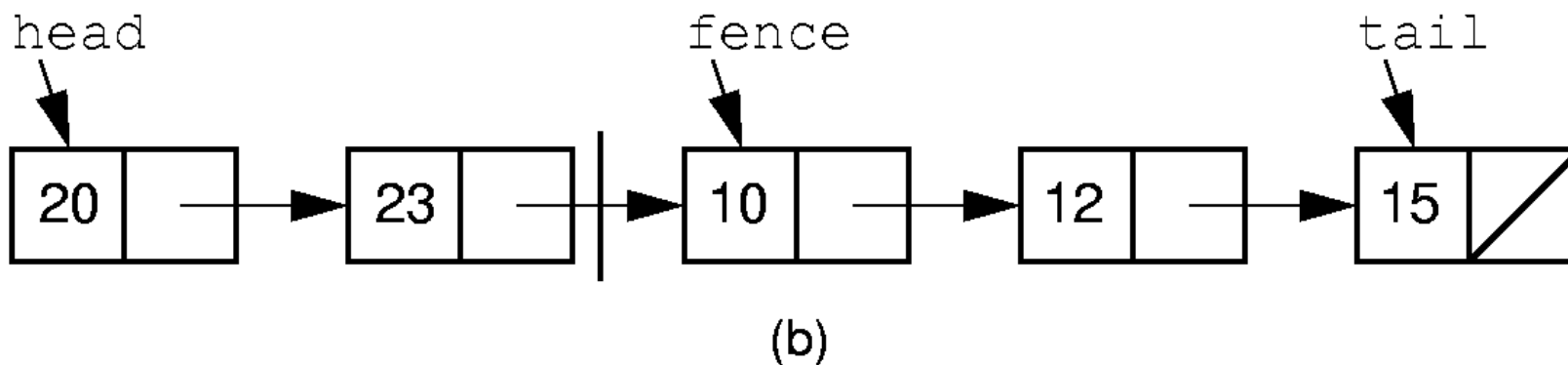
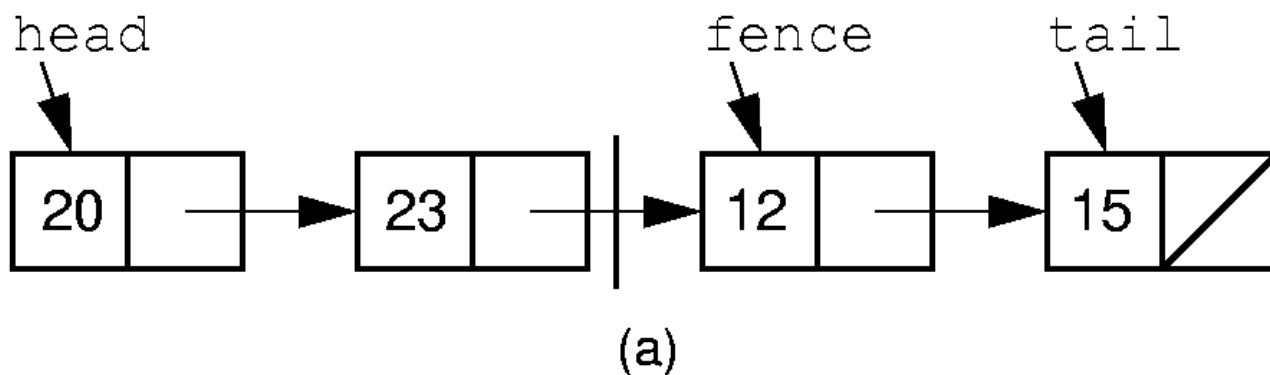
```
// Definition of an element of list
#define TRUE 1
#define FALSE 0
typedef int BOOL ;
typedef int element_type;
typedef struct node* node_ptr;
// Singly-linked list node
struct node
{
    element_type element;
    node_ptr next;
};
// Dynamic allocation of a new list element
node_ptr tmpNode = (node_ptr)malloc(sizeof(struct node));
```



例：在链表的当前操作位置(fence)插入一个新结点

Linked List Position (a)

Case a: fence指针指向右分区的第一个结点



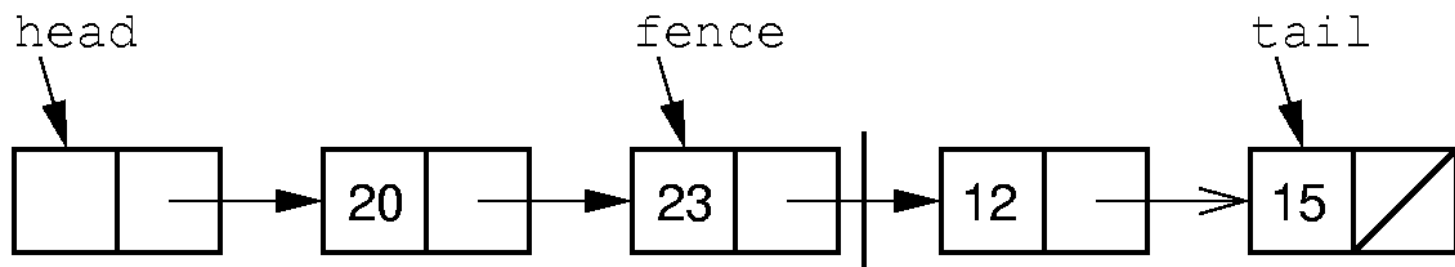


例：在链表的当前操作位置(fence)插入一个新结点

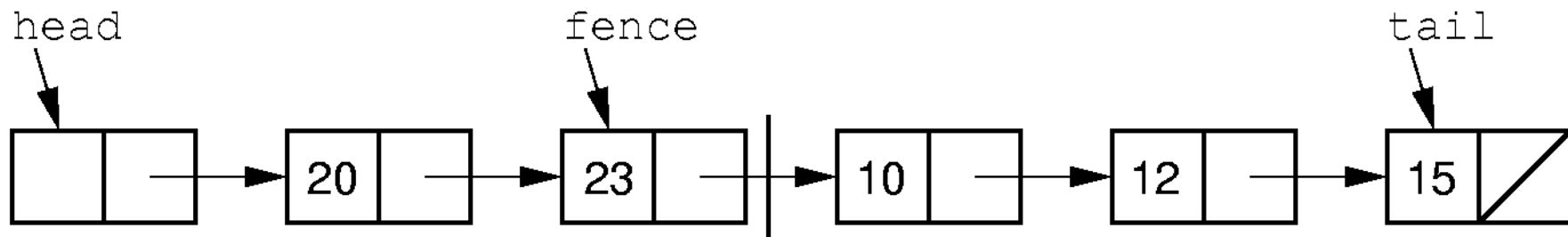
Linked List Position (b)

Case b: fence指针指向左分区的最后一个结点

这种更有效率！



(a)



(b)



Exe

- Given a singly linked list with head, which of the following statement signifies the list is empty

A .head==NULL B .head->next==NULL
C. head->next==head D. head!=NULL

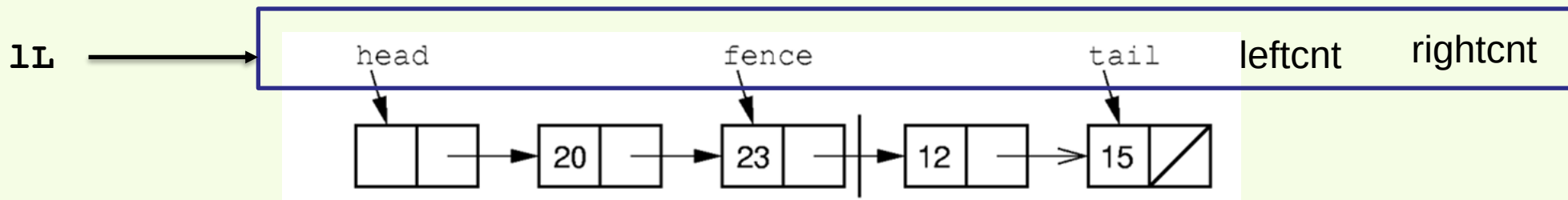
注：

- Head==null, 表示head无任何数据, 没有数据和next指针;
- head->next = null, 表示存在head数据, 但是指针是空, 表示链表中只有head一个结点。



Linked List (2)

```
// Linked list implementation
typedef struct LINKLIST *LList{
    node_ptr head; // Point to list header
    node_ptr tail; // Pointer to last
    node_ptr fence; // Last element on left
    int leftcnt;    // Size of left
    int rightcnt;   // Size of right
};
LList lL=(LList)malloc(sizeof(struct LinkLIST));
```





What is a List?

- In code: a collection of variable which provide the access to the list (beginning, current position) and related important information. It is pointed by one pointer to access it.
- A set of operation: insert, delete, prev(将当前位置往前挪), next(将当前位置往后挪)
- A data structure with one dimension (of growth or reduction)
- Notion of linear



Comparisons

Array-Based Lists:

- Insertion and deletion are $O(n)$.
- Prev and Direct access are $O(1)$.
- Array must be allocated in advance.
- No overhead (额外开销) if the array is not full.

Linked Lists:

- Insertion and deletion are $O(1)$.
- Prev and Direct access are $O(n)$.
- Space grows with number of elements.
- Every element requires overhead.



Linked List (2)

```
void init(LList listLink) {           // 初始化链表
    node_ptr headNode = (node_ptr)malloc(sizeof(struct node));
    headNode->element = 0;
    headNode->next = NULL;

    listLink->head = headNode;
    listLink->fence = listLink->tail = listLink->head;
    listLink->leftcnt = listLink->rightcnt = 0;
}

void removeall(LList listLink) {      // 从链表的第一个结点开始清空所有结点
    while(listLink->head != NULL) {
        listLink->fence = listLink->head;
        listLink->head = listLink->head->next;
        free(listLink->fence);
    }
}

void clear(LList listLink) {          // 清空所有结点后再初始化链表
    removeall(listLink);
    init(listLink);
}
```



Linked List (3)

```
void setStart(LList listLink) { // 将当前位置挪到头结点
    listLink->fence = listLink-> head;
    listLink->rightcnt += listLink->leftcnt;
    listLink->leftcnt = 0;
}
void setEnd(LList listLink) { // 将当前位置挪到尾结点
    listLink->fence = listLink->tail;
    listLink->leftcnt += listLink->rightcnt;
    listLink->rightcnt = 0;
}
void next(LList listLink) { // 将当前位置往右挪一位
    // Don't move fence if right empty
    if (listLink->fence != listLink->tail) {
        listLink->fence = listLink->fence->next;
        listLink->rightcnt--;
        listLink->leftcnt++;
    }
}
```



Linked List (4)

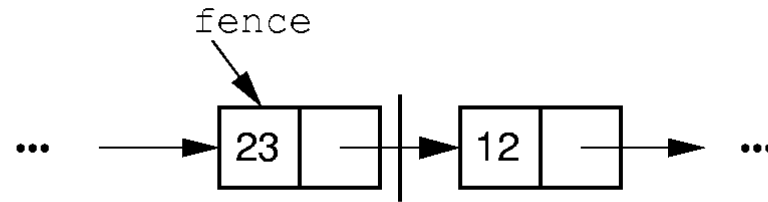
```
int leftLength(LList listLink)    // 读取左分区大小
{
    return listLink->leftcnt;
}

int rightLength(LList listLink)   // 读取右分区大小
{
    return listLink-> rightcnt;
}

BOOL getValue(LList listLink, element_type* it)
{
    // 读取当前位置下一个结点的值
    if(rightLength(listLink) == 0) return FALSE;
    *it = listLink->fence->next->element;
    return TRUE;
}
```

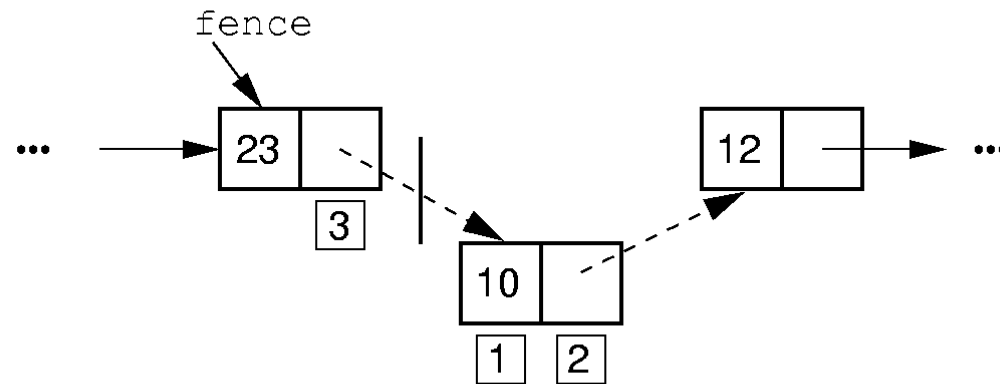


Insertion



Insert 10: [10 | -]

(a)



(b)



Insert

```
// 在右分区开始的地方插入新结点
// Insert at front of right partition
BOOL insert(LList listLink,  element_type item)
{
    // new a node
    node_ptr tmpNode = (node_ptr)malloc(sizeof(struct node));
    tmpNode->element = item;
    tmpNode->next = listLink->fence->next;
    listLink->fence->next = tmpNode;
    if (listLink->tail == listLink->fence)
        listLink->tail = listLink->fence->next;
    listLink->rightcnt++;
    return TRUE;
}
```



Append

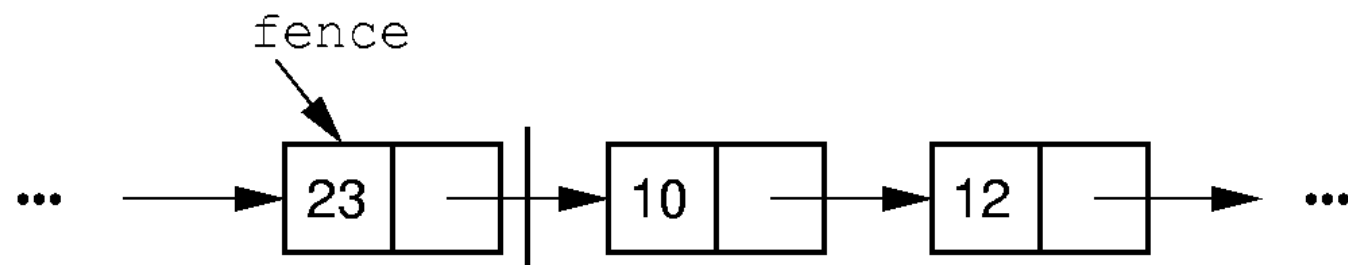
// 在链表的最后扩展一个结点

// Append Elem to end of the list

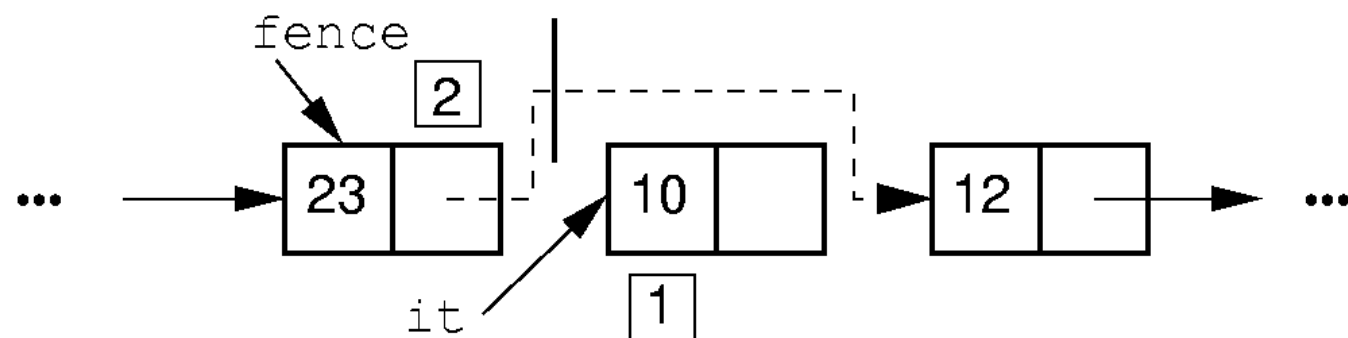
```
BOOL append(LList listLink,    element_type item) {  
    // new a node  
    node_ptr tmpNode = (node_ptr)malloc(sizeof(struct node));  
    tmpNode->element = item;  
    tmpNode->next = NULL;  
    listLink->tail->next = tmpNode;  
    listLink->tail = listLink->tail->next;  
    listLink->rightcnt++;  
    return TRUE;  
}
```



Remove



(a)



(b)



Remove

```
// 删除右分区第一个结点, 且获取该结点的值
// Remove and return first Elem in right partition
BOOL Remove(LList listLink,    element_type* it)
{
    if (listLink->fence->next == NULL) return FALSE;
    *it = listLink->fence->next->element; //Remember val
    // Remember link node
    node_ptr ltemp = listLink->fence->next;
    listLink->fence->next = ltemp->next; // Remove
    if (listLink->tail == ltemp) // Reset tail
        listLink->tail = listLink->fence;
    free(ltemp);    // Reclaim space
    listLink->rightcnt--;
    return TRUE;
}
```



Prev

```
// 将当前位置往前挪一位  
// Move fence one step left;  
// no change if left is empty  
void prev(LList listLink) {  
    node_ptr temp = listLink->head;  
    // No prev Elem  
    if (listLink->fence == listLink->head) return;  
    while (temp->next != listLink->fence)  
        temp=temp->next;  
    listLink->fence = temp;  
    listLink->leftcnt--;  
    listLink->rightcnt++;  
}
```



Setpos

// 将第pos个结点设为当前结点

// Set the size of left partition to pos

```
BOOL setPos(LList listLink, int pos) {  
    if ((pos < 0) || (pos > listLink->rightcnt+listLink->leftcnt))  
        return FALSE;  
    listLink->fence = listLink->head;  
    int i;  
    for( i=0; i<pos; i++)  
        listLink->fence = listLink->fence->next;  
    return TRUE;  
}
```



Print

```
// 打印整个列表
// print the list
void print(LList listLink) {
    node_ptr temp=listLink->head;
    printf("[");
    while (temp->next!= NULL){
        if (temp==listLink->fence){
            printf("| ");
        }
        printf("%d ", temp->next->element);
        temp=temp->next;
    }
    printf("]");
}
```



Summary

- Concept of List
 - Concept of “fence”
 - Implementation of “fence”
- Array-Based List
 - Access operation: $O(1)$
 - Insert, delete: $O(n)$
- Linked List
 - Structure node
 - Structure LINKLIST
 - Local operations ($O(1)$):
 - insert, delete, getValue...
 - Need going through the list ($O(n)$):
 - prev, print, find, setpos...